Static Analysis vs. Dynamic Analysis in Malware Detection – an AI Powered Comparison

Project Report by: Daniel Agranat, Daniel Buts and Daniel Ziv

Abstract – This work inspects the two leading malware analysis paradigms: static analysis and dynamic analysis. We use the power of AI to create a cybersecurity malware detection pipeline, to conduct a comparison and to help draw a conclusion regarding the proficiency of each approach and the proficiency of the two Boosting algorithms inspected, considering the static and dynamic artifacts of the same batch of benign and malicious executable files, provided in **quantitative form**.

Introduction

Static analysis and dynamic analysis are the two leading approaches in malware analysis. In Static analysis, an executable file is examined without executing it. This approach consists of the inspection of the Assembly machine instructions present in the disassembled binary image of the executable file.

Naturally, Static analysis has its own caveats, originating in various aspects. One of the prominent aspects is the difficulties in determining the correctness of the disassembly. Given the widespread presence of anti-analysis techniques, it is not always possible to assume that the disassembled image produced by the disassembler is reliable. Furthermore, the software can be obfuscated, and malicious portions of the code can be misinterpreted by the disassembler, potentially leading to a wrong conclusion about the nature of the file.

Dynamic Analysis is an observation-based technique. At its core stands the active execution of the inspected sample in a protected and isolated environments, often designated sandboxes and virtual machines, and observing the system's behavior.

Dynamic analysis is centered around monitoring the dynamic behavior of a system when a sample is executed on it: for example, if it downloads malicious payloads or proceeds to modify its own executable instructions or to modify the system's registry.

Like Static Analysis, dynamic analysis has its limitations, related mainly to the use of virtual machines and sandboxes to perform the analysis itself. The presence of a virtual machine can be detected by exploiting various execution tricks: the malware can calculate the time that elapses in the execution of certain operations, and if these were performed more slowly than expected, it can deduce consequently that the execution takes place on a virtual machine. In addition, execution in a secure environment can be detected through the execution of certain instructions at the hardware, for example, instructions that access CPU-protected registers. When the presence of a virtual machine is detected, malware can alter its behavior in some manner and avoid detection. With the high diffusion of malware, it is increasingly necessary to introduce AI solutions that allow rapid screening of potential threats, enabling SOC experts and malware analysts to respond in a timely and effective manner.

Methods

Dataset acquisition – the comparison between static analysis and dynamic analysis stands at the center of our project. Because of that, we were looking for a dataset that contains labeled entries both static and dynamic artifacts for the **same batch** of executable files, both malicious and benign. We were able to find such datasets in a repository maintained by the University of California, Irvine. The features in each dataset are of aggregative form, indicating the **number of** assembly commands and observed dynamic activities of each example.

Dataset adaptation – since the datasets are containing both dynamic and static features of both malicious and benign files, we split the datasets to two datasets representing each approach, containing the adequate features, and shuffled benign and malicious entries. After performing data cleansing and adaptation, a process which was EDA based – we had two model-ready datasets, one represents the static analysis approach and the other represents the dynamic analysis approach.

Choosing the learning algorithms – from the start, we decided to include a meaningful xAI element in our project. After exploring our options, we decided to inspect two Ensemble Learning Boosting algorithms: AdaBoost and GradientBoosting. The two chosen algorithms differ in their inner mechanisms of

classification: while the AdaBoost algorithm increases the weight of each misclassified example after each learner, so new learners can focus on the misclassified examples, the GradientBoosting algorithm fits each new learner to the negative gradient of the loss function. Although GradientBoosting is a generalization of AdaBoost, we wanted to see if the different training approaches will work in favor of one of them in our use case. Being Ensemble models that are based on decision trees, the chosen algorithms also return a confidence score along with their classification, indicating how certain the model is regarding the classification made for each example. The Boosting algorithm itself, by its nature, is serving as a "feedback loop".

Calibrating the models for the dynamic artifacts – After conducting a preliminary inspection of the performance of our models on each of the datasets, an observation was made that the Confidence Score on the dynamic artifacts dataset was too low, subsequently, the number of false positives was high and the need for model calibration emerged. To address this issue, we decided to implement Platt Scaling.

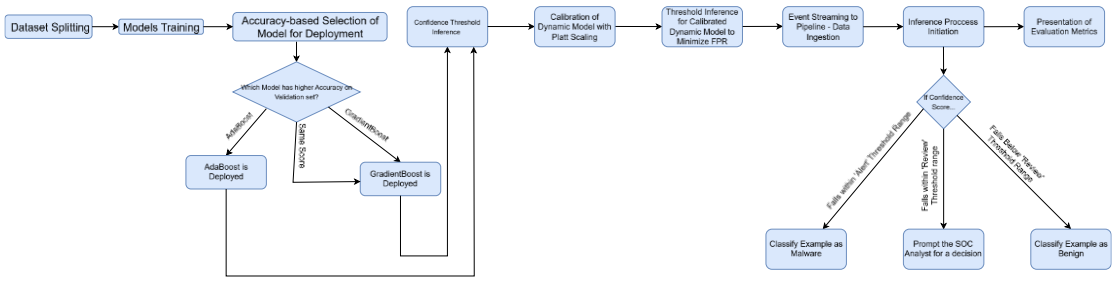
Platt Scaling is used when a classification model does not provide confidence score with its classifications or provide poor confidence score, that does not meet the requirements.

In Platt Scaling, the following mapping formula is used:

$$P(y = 1|x) = \frac{1}{1 + e^{Af(x)+B}}$$

This formula is used to convert the original confidence estimates of the model to calibrated probabilities where A and B are two parameters that are learned on the validation set, and $f(x)$ is the confidence score given by the original model.

Creation of Detection Pipeline – the workflow of our pipeline is depicted in the following diagram



Results

AdaBoost vs. GradientBoosting – the two inspected learning algorithms achieved the same Accuracy of 0.8311, and so, according to the diagram above, GradientBoosting was deployed as the learning algorithm of the pipeline. The difference between the training of each algorithm did not have any effect in our use case, considering the amount of data we were using, its attribution to the same batch of executable files and its quantitative nature.

Static Analysis vs Dynamic Analysis – Before inspecting the performance of the deployed model on each dataset individually, we noticed that the prediction agreement in the test set is 83.1%, meaning the model agrees on the classification it has made, both using the static artifacts dataset and the dynamic artifacts data separately on 83.1% of the new static and dynamic examples that were provided to it. This is a meaningful result, especially when considering the difference in the number of features in each dataset.

When inspecting the performance of the model on each of the datasets, it can be observed that the evaluation metrics for the static dataset are perfect, resulting in 1s across the board. Initially thinking our model overfits, we performed a few standard overfitting tests and discovered that our model does not overfit, but because our static dataset is very feature-reach, its feature space allows near perfect separation, which results in such high evaluation metrics.

On the dynamic artifacts dataset, the model demonstrates normal behavior, considering the relatively small number of dynamic features, achieving a macro average f1 score of ~0.5.

Further discussions, including graphs, diagrams and charts can be found in the attached clip and in the project's repository on GitHub